C11 / POSIX · SCHEDULING · CONCURRENCY

MINI CPU SCHEDULER

Project overview & technical report

A code-grounded overview of scheduling algorithms, synchronization, inter-process communication, performance metrics, and observed outputs.

PROJECT PARTNERS

Hamza Faheem

Mutasim ur Rehman

INSIDE THE REPORT

01 — 03 Overview, theory & algorithms

04 — 06 Runtime, source structure & metrics

07 — 08 IPC, features & limitations

01 PROJECT OVERVIEW

What the scheduler does

Mini CPU Scheduler is a C11/POSIX simulation of a multiprogramming CPU scheduler. It reads jobs from a text file, models arrivals and CPU bursts in discrete time units, executes one unit at a time through worker threads, and reports per-job and aggregate performance. The scheduling policy, synchronization protocol, logging, and inter-process communication remain visible in the source code.

Runtime architecture

Main / Dispatcher	Arrival thread	Job threads
Selects next job Advances time	Enqueues jobs At each tick	One thread / job Runs one unit

↳ Logger child · pipe + fork

Writes event lines to `schedule.log`.

↳ Report child · fork + exec

Reads and publishes shared metrics.

Core goals

- Compare FCFS, SJF, SRTF, Round Robin, and preemptive priority scheduling.
- Demonstrate thread coordination using pthreads, mutexes, and POSIX semaphores.
- Demonstrate process coordination using pipes, `fork/exec`, POSIX shared memory, and a named semaphore.
- Produce Gantt charts, per-job metrics, comparison tables, event logs, and a final report process.

Build and run

The Makefile builds `scheduler` and `report` with GCC using `-std=c11 -Wall -Wextra -pthread -O2` and links `-lrt`.

```
$ make
$ ./scheduler inputs/jobs1.txt all 2 --quiet
$ ./scheduler inputs/prio.txt prio --aging 4
```

One simulated time unit is 200 ms by default. Override at compile time with `-DTIME_UNIT_MS=50`.

02 THEORETICAL BACKGROUND

How CPU scheduling is evaluated

CPU scheduling decides which ready process or thread receives the processor next. A scheduler balances competing objectives: low waiting time, low turnaround time, fast first response, fairness, and low context-switch overhead. No single policy optimizes all objectives, so the project implements several policies for comparison.

Key terms and definitions

Term	Definition
Arrival time	Time at which a job becomes ready.
Burst time	Total CPU time required by the job.
Completion time	Time at which the job finishes.
Turnaround time	Completion − arrival.
Waiting time	Turnaround − burst.
Response time	First CPU allocation − arrival.
Context switch	A direct change from one job to a different job. The first dispatch and transitions to or from IDLE are not counted.

Scheduling trade-offs

NON-PREEMPTIVE

Avoids interrupting a running job, usually reducing switching, but can delay short jobs behind a long burst.

PREEMPTIVE

Reconsiders the queue each time unit, improving responsiveness but potentially increasing context switches and overhead.

> These trade-offs are measured using average waiting, turnaround, and response times, context-switch counts, and total simulated time.

03 SCHEDULING ALGORITHMS

Selection policies · Part I

At each simulated time unit, the arrival thread enqueues newly available jobs. The dispatcher applies the selected policy to the ready queue. A selected job executes one CPU unit, after which the scheduler updates its remaining time and metrics.

FCFS — First Come, First Served

NON-PREEMPTIVE

FCFS selects the head of the ready queue. Jobs run to completion in arrival order. It is simple and fair by arrival order, but a long first job can create the convoy effect and increase response time for short jobs.

Arrival enqueue tail → **Ready queue** take head → **CPU** run to finish → **Next** repeat

SJF — Shortest Job First

NON-PREEMPTIVE

SJF scans the ready queue and selects the job with the smallest total burst. Ties go to earlier arrival, then lexicographic job ID. Once a job starts, it retains the CPU until completion.

Ready queue inspect bursts → **Select** minimum burst → **CPU** run to finish → **Next** rescan

COMPARISON AT A GLANCE

FCFS uses **arrival order**; SJF uses **total burst length**. Neither interrupts a job already running. Their performance can be identical on workloads where the first job starts before shorter jobs arrive.

03 SCHEDULING ALGORITHMS

Selection policies · Part II

SRTF — Shortest Remaining Time First

PREEMPTIVE

At each time unit SRTF compares the running job with the ready queue and chooses the smallest remaining burst. A tie favors the current job. Short jobs can respond quickly, but long jobs may starve if shorter jobs keep arriving.

Tick new arrivals → **Compare** remaining time → **Preempt?** switch if shorter → **CPU** run one unit

Round Robin

TIME-SLICED

Round Robin uses a FIFO queue and a configurable quantum `q`. A job runs for at most `q` units before moving to the tail, provided another job waits. If none waits, it continues. This improves fairness and response at the cost of more context switches.

Queue take head → **Quantum** run q units → **Expire** move to tail → **Repeat** fair sharing

Priority scheduling with aging

PREEMPTIVE

Smaller numbers mean higher priority. The dispatcher selects the lowest effective priority, breaking ties by arrival and then job ID. Optional aging improves a waiting job's effective priority by 1 after every `N` waiting units, down to 0; the boost resets when the job receives CPU time.

Wait accumulate units → **Age** −1 per N units → **Select** lowest effective → **CPU** reset boost

Starvation risk: SRTF can postpone long jobs; priority aging is included to reduce the likelihood of indefinite postponement.

04 CONCURRENCY MODEL

Runtime control & synchronization

The dispatcher advances the simulated clock one unit at a time. Semaphores coordinate the execution so exactly one job thread is released for a CPU unit; the dispatcher then waits until that job signals completion.

One simulated CPU tick



Thread and process responsibilities

Component	Responsibility
Dispatcher / main	Posts `tick`, waits for arrivals, selects a job, posts `run[i]`, waits on `done`, and updates metrics.
Arrival thread	Waits on `tick`, enqueues newly arrived jobs, then posts `arrivals_done`.
One thread per job	Waits on `run[i]`, simulates a unit with `nanosleep`, then posts `done`.
Logger process	Reads event lines from a pipe and writes them to `schedule.log`.
Report process	Waits on the named ready semaphore, maps shared memory, prints final metrics, and exits.

Synchronization primitives

- `queue_mtx` protects the ready queue, mutable job fields, Gantt trace, counters, and logger writes.
- `print_mtx` serializes console output to prevent interleaving.
- Unnamed POSIX semaphores coordinate `tick`, `arrivals_done`, `run[i]`, and `done`.
- On teardown, all worker threads are joined, semaphores destroyed, and logger/report children reaped with `waitpid`.

05 REPOSITORY MAP

File structure & responsibilities

The source is organized into focused modules for scheduling, input parsing, output, logging, and shared-memory reporting.

Path	Responsibility
`Makefile`	Builds `scheduler` and `report`; provides `run`, `results`, `csdemo`, `agingdemo`, and `clean`.
`README.txt`	Documents commands, assumptions, synchronization, bonus features, and limitations.
`src/common.h / common.c`	Shared constants, enums, structs, algorithm names, and error helper.
`src/jobs.c / jobs.h`	Parses and validates inputs; resets per-run job state.
`src/sched.c / sched.h`	Ready queue, scheduling policies, threads, dispatcher, Gantt trace, and metrics.
`src/output.c / output.h`	Prints Gantt charts, metrics tables, averages, and comparisons.
`src/logger.c / logger.h`	Creates pipe and forked logger child; writes timestamped events to `schedule.log`.
`src/shm.c / shm.h`	Creates POSIX shared memory / named semaphore; forks and execs report; cleans IPC objects.
`src/report.c`	Separate report executable reading shared metrics and printing the final report.
`inputs/*.txt`	Sample workloads for normal, priority, starvation, and comparison demos.
`results.txt`	Generated comparison tables for all sample inputs.

INPUT FORMAT

```
id arrival burst [priority]
```

Each non-comment line follows this layout. Blank lines and `#`-prefixed lines are ignored. The parser rejects malformed or missing fields, negative arrival times, bursts below 1, duplicate IDs, and workloads above 64 jobs.

06 EVALUATION

Outputs & performance metrics

Each run emits a Gantt chart, per-job metrics, averages, context-switch count, total simulated time, and event log. In `all` mode, the comparison table places the policies side by side. A separate report process prints final metrics from a shared-memory snapshot.

Metric formulas

Turnaround

`completion − arrival`

Waiting

`turnaround − burst`

Response

`first CPU − arrival`

Average

`sum(job values) / job count`

Verified sample — inputs/jobs1.txt, quantum 2

Algorithm	Avg waiting	Avg turnaround	Avg response	Context switches
FCFS	3.33	8.67	3.33	2
SJF	3.33	8.67	3.33	2
SRTF	3.00	8.33	2.00	3
Round Robin (q=2)	6.00	11.33	1.00	7

Interpretation of the sample

Round Robin delivers the fastest average response because jobs receive CPU time early, but it also produces the most context switches and highest average waiting time. SRTF slightly improves waiting and response by favoring shorter remaining bursts. FCFS and SJF match here because the first job occupies the CPU before shorter jobs become available.

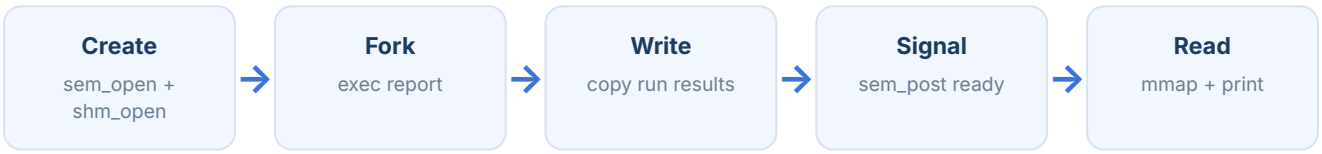
1.00 **Round Robin average response**
Fastest response in the verified sample, traded against seven context switches.

07 IPC & REPORTING

Shared-memory report handoff

The reporting path runs in a separate process. The scheduler creates a named semaphore `/mini_sched_ready` initialized to 0 and a POSIX shared-memory object `/mini_sched_shm` sized for one `shm_report_t`, then forks and execs `./report`.

Publication sequence



Data layout

The `shm_report_t` structure contains the number of runs, aging value, input filename, and an array of `run_result_t`. Every run result contains the algorithm, quantum, context-switch cost, job count, per-job metrics, averages, context-switch count, total time, and validity flag.

Why the semaphore matters

The report child starts before the parent finishes filling shared memory. Its initial blocking wait prevents reading partially written data. After `sem_post`, the mapping contains a consistent snapshot the child can print. The parent waits for the child, unmaps and closes the segment, and unlinks the shared-memory object and named semaphore.

CLEANUP AND SAFETY

- Stale IPC objects are unlinked before creation to recover from a previous crashed run.
- IPC names are fixed, so only one scheduler instance should run at a time.
- The report child exits independently but is reaped with `waitpid` before cleanup.

08 FEATURES & CONSTRAINTS

Bonus features, limits & takeaways

Context-switch cost

The optional `--cs N` flag charges `N` simulated overhead units whenever the CPU changes directly from one job to another. The Gantt chart labels these periods `CS`. Waiting, turnaround, completion, and total time include the added cost.

Aging and starvation

With `--aging N`, each `N` units that a ready job waits improves its effective priority by 1, but never below 0. The starvation workload illustrates how a long job can be postponed by new short jobs under SJF or SRTF, while priority aging improves fairness.

Known limitations

- Maximum of 64 jobs; job IDs allow 15 characters plus the terminating null byte.
- Arrivals during context-switch overhead are enqueued after that overhead unit.
- The simulated clock is discrete; each CPU unit uses `nanosleep`.
- Only one scheduler instance can run safely at a time because IPC object names are fixed.
- Standard C and POSIX APIs only; no third-party package dependencies.

Closing summary

This project joins classic scheduling theory with an executable concurrency model. Gantt segments expose scheduling decisions; semaphores reveal coordination; logs record events; and the shared-memory report demonstrates a clean transfer from the scheduling process to a reporting process.